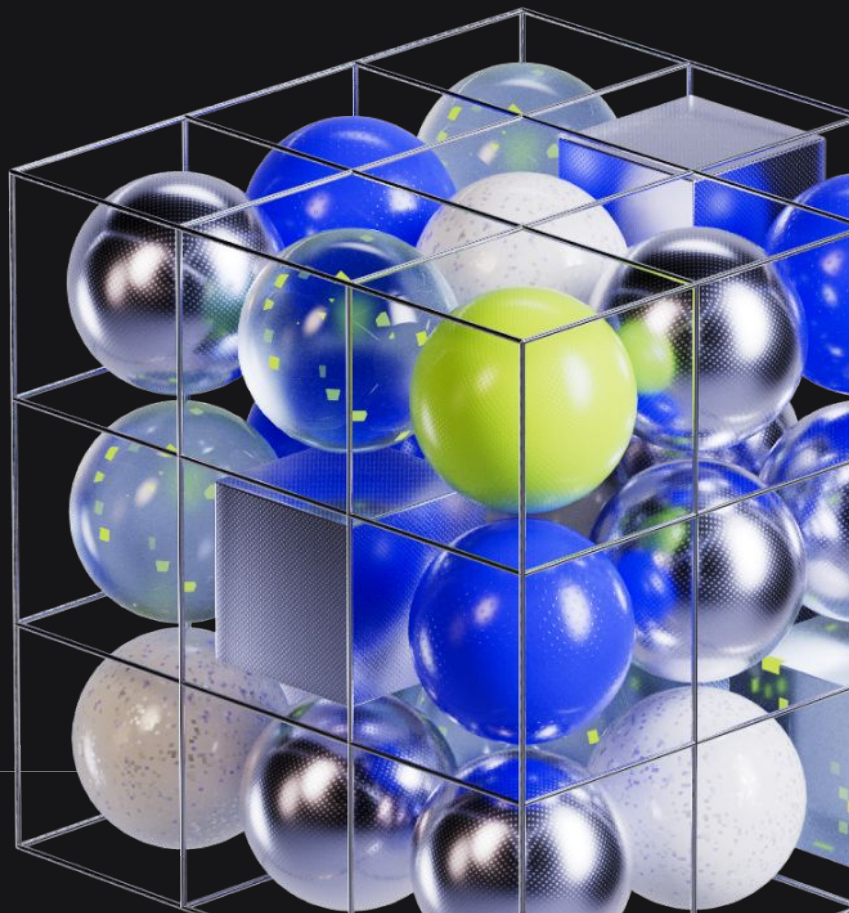


# TECHNICAL INTERVIEW

ANDROID



## Introduction to technical conversation interview

A 1-hour in-depth technical discussion with 1-2 engineers, focused on exploring your skills and experience. It includes a short coding exercise and covers topics ranging from computer science fundamentals to data structures and algorithms.

## Evaluation criteria

The interview evaluates four main areas. You'll be assessed on writing clean, modular code, creating meaningful tests with solid coverage, and building resilient UIs that handle loading, errors, and edge cases smoothly.

## Structure of the interview

### 1 Introduction

- Brief introduction of yourself and your experience
- Interviewers introduce themselves and provide context

### 2 Kotlin & Android Fundamentals

- Demonstrate understanding of core Kotlin syntax and constructs (e.g., data classes, sealed classes, extension functions)
- Discuss practical application in Android development
- Explain Android component lifecycles and common challenges

### 3 Asynchronous Programming & Concurrency

- Deep dive into Kotlin Coroutines: Dispatchers, suspend functions, Job lifecycle, structured concurrency
- Discuss reactive programming using Flow and compare with LiveData
- Explain thread safety, synchronization strategies, and handling concurrency in multithreaded environments

## Structure of the interview

### 4 Architecture & Design Patterns

- Describe commonly used architecture patterns (MVVM, MVI, Clean Architecture)
- Break down responsibilities across layers: UI, ViewModel, Repository, Data Source, Use Cases
- Discuss Dependency Injection principles and benefits (e.g., modularity, testability)

### 5 Testing

- Explain unit testing practices for Android
- Discuss testing ViewModels and Use Cases using tools like Mockito
- Demonstrate understanding of writing testable and maintainable code

### 6 UI Optimization & Performance

- Discuss RecyclerView performance improvements
- Explain layout performance strategies

### 7 Problem-Solving & Autonomy

- Share your approach to debugging and tackling complex issues
- Discuss thinking through edge cases, failures, and error handling
- Demonstrate proactive communication, independent decision-making, and ownership during challenges

### 8 Wrap-up & Questions

- Opportunity to ask the interviewer questions
- Final thoughts and feedback

## Preparation recommendations

We recommend that you refresh your knowledge of Kotlin fundamentals and data structures, and to demonstrate a solid understanding of testing and multithreading. To support this, we encourage you to check the following articles and resources.

### Articles & Resources

[Architecture Coroutines](#)

[Concurrent Map](#)

[Recycler View](#)

[Testing Fundamentals](#)

[The Fundamentals of Android at Revolut](#)

### Tips

- Attention to the time management, you need to cover as many areas as possible
- Clarify requirements and ask relevant follow-up questions
- Make reasonable assumptions where needed
- Communicate your thought process clearly
- Focus on correctness, simplicity, and performance
- Show autonomy while remaining collaborative
- Be proactive about handling edge cases and potential issues

### Key takeaways

- Master Kotlin fundamentals, Android lifecycles, and architectural patterns
- Gain hands-on experience with Coroutines, Flow, and concurrency handling
- Prepare to discuss and test your knowledge of UI performance and optimization
- Practice clear, structured communication during problem-solving
- Be ready to demonstrate independence and decision-making in a real-world scenario

By following this guide, you will be well-prepared to tackle Revolut's Technical Conversation interview confidently. Good luck!

**WE HOPE THIS GUIDE HAS  
BEEN USEFUL TO YOU.  
SEE YOU SOON?**

